

Room XI Connect - Critical Fixes & Patches

Overview

This document provides step-by-step instructions for applying critical fixes to resolve production blockers.

Fix #1: TypeScript Server Configuration

Problem

Server-side TypeScript files cannot be imported with `.ts` extensions, causing build failures.

Solution

File: `tsconfig.server.json`

Replace the entire file with:

JSON

```
{
  "extends": "../tsconfig.node.json",
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "noEmit": true,
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noImplicitAny": true,
    "strict": true,
    "esModuleInterop": true,
    "moduleResolution": "bundler"
  },
  "include": ["server/**/*.ts", "server/**/*.tsx"],
  "exclude": ["node_modules", "dist"]
}
```

Verification

Bash

```
npm run typecheck:server  
# Expected output: No errors
```

Fix #2: ESLint Configuration

Problem

ESLint scans mobile build artifacts and node_modules, causing extremely slow linting and false positives.

Solution

File: `eslint.config.js`

Update the ignores array at the top of the export:

JavaScript

```
import tseslint from "typescript-eslint";  
  
export default tseslint.config(  
  {  
    ignores: [  
      "dist/**",  
      "node_modules/**",  
      "android/**",  
      "ios/**",  
      ".next/**",  
      ".vercel/**",  
      "build/**",  
      ".git/**"  
    ],  
  },  
  // ... rest of your eslint config  
);
```

Verification

Bash

```
npm run lint
```

```
# Expected: Completes in <30 seconds with no errors
```

Fix #3: Dependency Vulnerabilities

Problem

12 vulnerabilities (2 HIGH, 10 MODERATE) in dependencies pose security risks.

Solution

Step 1: Run automated fixes

Bash

```
npm audit fix
```

Step 2: For breaking changes, manually update

Bash

```
# Update Vite (fixes esbuild vulnerability)
npm install vite@latest --save-dev

# Update web-push (fixes jws vulnerability)
npm install web-push@latest --save

# Update query-string (fixes qs vulnerability)
npm install query-string@latest --save
```

Step 3: Update package.json if needed

If the above commands don't resolve all vulnerabilities, manually edit `package.json` :

JSON

```
{
  "dependencies": {
    "nodemailer": "^7.1.0",
    "web-push": "^3.6.7"
  },
  "devDependencies": {
    "vite": "^5.4.8"
  }
}
```

Then run:

```
Bash
```

```
npm install
```

Verification

```
Bash
```

```
npm audit  
# Expected output: 0 vulnerabilities
```

Fix #4: Production Environment Configuration

Problem

Critical secrets are missing from production environment, breaking authentication and features.

Solution

Step 1: Generate required secrets

```
Bash
```

```
# Generate SESSION_SECRET (32 random characters)  
openssl rand -base64 32  
  
# Generate ENCRYPTION_SECRET (32 random characters)  
openssl rand -base64 32  
  
# Generate VAPID keys for push notifications  
npx web-push generate-vapid-keys
```

Step 2: Create `.env.production` file (DO NOT COMMIT)

```
Bash
```

```
# Database Connection  
DATABASE_URL=postgresql://[user]:[password]@[host]:[port]/[database]  
  
# Session Management (from openssl command above)
```

```
SESSION_SECRET=[generated-32-char-secret]

# Encryption (from openssl command above)
ENCRYPTION_SECRET=[generated-32-char-secret]

# Email Configuration (choose one)
EMAIL_PROVIDER=sendgrid
SENDGRID_API_KEY=SG.[your-sendgrid-api-key]

# OR use Gmail
# EMAIL_PROVIDER=gmail
# GMAIL_USER=your-email@gmail.com
# GMAIL_PASSWORD=your-app-password

# Push Notifications (from web-push command above)
VAPID_PUBLIC_KEY=[generated-public-key]
VAPID_PRIVATE_KEY=[generated-private-key]
VAPID_EMAIL=mailto:support@roomxiconnect.org

# Admin Credentials (password should be bcrypt hashed)
ADMIN_USERNAME=admin
ADMIN_PASSWORD=$2b$10$[bcrypt-hashed-password]

# Node Environment
NODE_ENV=production
LOG_LEVEL=info

# Optional: OpenAI API (if using AI features)
OPENAI_API_KEY=sk-[your-openai-key]
```

Step 3: Deploy to Vercel

Bash

```
# Set environment variables in Vercel dashboard
vercel env add DATABASE_URL
vercel env add SESSION_SECRET
vercel env add ENCRYPTION_SECRET
vercel env add SENDGRID_API_KEY
vercel env add VAPID_PUBLIC_KEY
vercel env add VAPID_PRIVATE_KEY
vercel env add VAPID_EMAIL

# Or use Vercel CLI to import from .env.production
vercel env pull .env.production
```

Step 4: Deploy backend

Bash

```
# If using Supabase Edge Functions
supabase functions deploy

# If using custom server
npm run build
npm run start
```

Verification

Bash

```
# Test locally
NODE_ENV=production npm run start

# Check health endpoint
curl http://localhost:5000/api/health
# Expected: { "status": "ok" }

# Test in production
curl https://roomxi.live/api/health
# Expected: { "status": "ok" }
```

Fix #5: Post-Signup UX Issue

Problem

Users are redirected to `/home` after signup, but the page shows a loading spinner indefinitely.

Root Cause Analysis

This is likely caused by one of:

1. Backend API not returning user session data
2. Database query timing out
3. CORS issue preventing API calls
4. Missing environment variables on live server

Solution

Step 1: Check server logs

Bash

```
# SSH into production server
ssh user@roomxi.live

# Check application logs
tail -f /var/log/roomxi-connect/app.log

# Look for errors like:
# - "DATABASE_URL not set"
# - "Cannot connect to database"
# - "CORS error"
# - "Session middleware error"
```

Step 2: Verify database connection

Bash

```
# From server, test database
psql $DATABASE_URL -c "SELECT 1;"
# Expected: (1 row )
```

Step 3: Check API endpoint

Bash

```
# Test the user profile endpoint
curl -H "Authorization: Bearer $TOKEN" https://roomxi.live/api/auth/me

# Should return user data, not error
```

Step 4: Temporary workaround (while debugging)

Edit `src/pages/auth/SignupComplete.tsx` :

TypeScript

```
// Change redirect from /home to /explore
useEffect(() => {
  // Redirect to explore instead of home while debugging
  setTimeout(() => {
    navigate("/explore");
  }, 1500);
}, [navigate]);
```

Step 5: Permanent fix

Once root cause is identified:

- If database issue: Verify DATABASE_URL and run migrations
- If CORS issue: Check CORS middleware configuration
- If session issue: Verify SESSION_SECRET is set
- If API issue: Check server logs for specific errors

Verification

Bash

```
# Test signup flow end-to-end
1. Go to https://roomxi.live/auth/signup
2. Complete all steps
3. Should redirect to /home (or /explore ) and load content
4. Should NOT show loading spinner for >5 seconds
```

Fix #6: Secure Admin Credentials

Problem

Admin password is stored as plaintext in `.env` file, which is a security risk.

Solution

Step 1: Generate bcrypt hash

Bash

```
# Using Node.js
node -e "console.log(require('bcryptjs').hashSync('your-admin-password',
10))"

# Or using online tool (NOT recommended for production)
# https://bcrypt-generator.com/
```

Step 2: Update `.env.production`

Bash


```
ADMIN_PASSWORD=$2b$10$[bcrypt-hash-from-step-1]
```

Step 3: Remove `.env` from git history

Bash

```
# Add to .gitignore
echo ".env" >> .gitignore
echo ".env.production" >> .gitignore
echo ".env.local" >> .gitignore

# Remove from git history
git rm --cached .env
git commit -m "Remove .env from version control"
```

Step 4: Verify admin login works

Bash

```
# Test admin panel login
curl -X POST https://roomxi.live/api/admin/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"your-admin-password"}'

# Expected: { "token": "..." }
```

Fix #7: Database Migrations

Problem

Database schema may not be up-to-date on production server.

Solution

Step 1: Generate migration

Bash

```
# If using Drizzle ORM
npm run db:generate

# This creates a new migration file in supabase/migrations/
```

Step 2: Apply migration

Bash

```
# Push to production database  
npm run db:push
```

```
# Or if using Supabase  
supabase db push
```

Step 3: Verify schema

Bash

```
# Connect to production database  
psql $DATABASE_URL
```

```
# List tables  
\dt
```

```
# Check schema  
\d users  
\d moods  
\d programs
```

Fix #8: Enable Monitoring & Error Tracking

Problem

Production errors are not being tracked or monitored.

Solution

Step 1: Set up Sentry

Bash

```
npm install @sentry/react @sentry/tracing --save
```

Step 2: Configure in `src/main.tsx`

TypeScript

```
import * as Sentry from "@sentry/react";

Sentry.init({
  dsn: import.meta.env.VITE_SENTRY_DSN,
  environment: import.meta.env.MODE,
  tracesSampleRate: 1.0,
} );
```

Step 3: Set environment variable

Bash

```
# In Vercel dashboard
VITE_SENTRY_DSN=https://[your-sentry-dsn]@sentry.io/[project-id]
```

Step 4: Test error tracking

Bash

```
# Trigger a test error
Sentry.captureException(new Error("Test error" ));

# Should appear in Sentry dashboard
```

Deployment Checklist

After applying all fixes, verify:

- ☐ `npm run typecheck` passes
- ☐ `npm run typecheck:server` passes
- ☐ `npm run lint` completes in <30 seconds
- ☐ `npm run build` succeeds
- ☐ `npm audit` shows 0 vulnerabilities
- ☐ All environment variables set in production
- ☐ Database migrations applied
- ☐ Health check endpoint returns 200
- ☐ Signup flow completes without errors
- ☐ User can access dashboard after signup

- ☐ Admin panel accessible
 - ☐ Error tracking working
 - ☐ Monitoring alerts configured
-

Rollback Plan

If issues occur after deployment:

Bash

```
# Revert to previous Vercel deployment
vercel rollback

# Or manually rollback
git revert [commit-hash]
git push

# If database issue, restore from backup
# Contact Supabase support for point-in-time recovery
```

Support & Debugging

Common Issues

Issue: "DATABASE_URL is not set"

Bash

```
# Solution: Set DATABASE_URL in environment
vercel env add DATABASE_URL
```

Issue: "SESSION_SECRET is not set"

Bash

```
# Solution: Generate and set SESSION_SECRET
openssl rand -base64 32
vercel env add SESSION_SECRET
```

Issue: "CORS error when calling API"

Bash

```
# Solution: Check CORS middleware in server/middleware/cors.ts  
# Ensure frontend URL is whitelisted
```

Issue: "Post-signup page stuck loading"

Bash

```
# Solution: Check server logs  
# Verify database connection  
# Check API response from /api/auth/me
```

Getting Help

1. Check server logs: `tail -f /var/log/roomxi-connect/app.log`
2. Check Sentry for errors: <https://sentry.io/>
3. Check Vercel deployment logs: <https://vercel.com/dashboard>
4. Check database status: Supabase dashboard
5. Contact DevOps team for infrastructure issues

Last Updated: January 6, 2026

Status: Ready for Implementation